NOTE **Choosing the best hyperparameters with gradient descent**

This flexibility in modeling gives rise to a natural question: how to appropriately set the values for these hyperparameters. Luckily, we already know a good way of setting the hyperparameters of a GP model. It is by choosing the values that explain our data the best, or in other words, by maximizing the marginal log likelihood, specifically via gradient descent.

Just like before when we wanted to optimize the hyperparameters of the mean function, we now do this by simply passing the variables that we'd like to optimize—the parameters of the covariance function—to Adam:

```
optimizer = torch.optim.Adam(model.covar_module.parameters(), lr=0.01)
```

By running gradient descent, we can arrive at good values for these parameters. Specifically, I obtained a length scale of roughly 1.3 and an output scale of roughly 2.1. That is, to fit the three-point training data set well, we want the GP to be lightly smoother (with a length scale greater than 1) and we also want the range of our predictions to be larger (with a larger output scale).

This is certainly a reassuring result, as our objective function does have a wide range of values—at input 3, it takes on a value of -2, which is well outside of the CI with an output scale of 1.

3.4.2 Controlling smoothness with different covariance functions

Thus far, we have exclusively used the RBF kernel as our covariance function. It is, however, entirely possible to use a different kernel for our GP if RBF is not appropriate. In this subsection, we will learn to use another family of kernels, the Matérn kernel, and see what effect this kernel would have on our GP.

NOTE **By using a Matérn kernel, we are specifying the smoothness of the function our GP models. Smoothness is a technical term here that refers to the differentiability of the function; the more times a function is differentiable, the smoother it is. We can roughly think of this as how much the function values "jump" up and down in a jagged manner.**

The RBF kernel models functions that are *infinitely* differentiable, which is a property that not many functions in the real world has. Meanwhile, a Matérn kernel produces functions that are finitely differentiable, and exactly how many times these functions may be differentiated (that is, how smooth these functions are) is controlled by a settable parameter, as we will see shortly.

To see the Matérn kernel in action, we first re-implement our GP model class:

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>